

Föreläsning 3

Ändrad
22 Sep
01:10

-- scanf och variabelnamn

Exemplen hittills har varit trista i ett avseende: de har varit ödesbestämda från start till mål. Användaren har inte kunnat ingripa i förloppet. Nu ska vi ändra på den saken.

Det hela är visserligen tämligen primitivt, ingen grafik, inga knappar att klicka på, ingen mus att hytta med. Det enda användaren kan göra är att mata in text via tangentbordet.

Texten som användaren matar in lagras i en variabel. Det går till på följande sätt:

```
1  #include <stdio.h>
2
3  main()
4  {
5      int year, month, day;
6
7      printf("Ange år: ");
8      scanf("%d", &year);
9
10     printf("Ange månad och dag: ");
11     scanf("%d %d", &month, &day);
12
13     printf("Personnummer: %d %d %d\n", year % 100, month, day);
14 }
```

Funktionen som läser in värden heter `scanf`.

Anropet till `scanf` liknar anropet till `printf`. Först kommer en sträng (en text inom citationstecken), därefter några variabelnamn separerade med kommatecken. Det är förstås dessa variabler vi läser in värden till. Specifikationen `%d` anger som förut att det är heltal vi läser in till.

Lägg märke till `&` som står framför variablerna.

Jag kan inte låta bli att säga "adressen till" när jag ser `&`. För det är vad tecknet betyder i detta sammanhang.

Funktionen `scanf` vill veta var de inmatade värdena ska läggas, på vilken plats i minnet. Operatören `&` plockar fram adressen.

Glömmer du `&` kan det hända att variabelns *värde* används som adress. Då blir det fel...

Såhär ser det ut på bildskärmen när programmet körs:

```
1  Ange år: 1963
2  Ange månad och dag: 08 06
3  Personnummer: 63 8 6
```

En del av texten skrevs av datorn. En del av användaren. Användarens text är *markerad*.

Det som händer när programmet körs är att talen som användaren matar in hamnar i variablerna. Det bekräftas av utskriften på rad 3.

Det finns en liten finess i utskriften av året. Jag dividerar bort de två första siffrorna med hjälp av operatoren `%`. Det är en heltalsoperation. Funktionen brukar kallas *modulo*. Den tar resten efter division. $1963/100 = 19 + 63/100$ -- resten är 63.

Däremot misslyckas jag kapitalt med att få datumet på den vanliga formen, 630806.

Här är ett program som klarar datumets form bättre. Programmet använder specifikationen `%2.2d` för att skriva ut de tre talen. Första tvåan betyder två positioner. Andra tvåan betyder två siffror. Jag hittade möjligheten i en [manual](#).

Flyttal: float och double

Det finns två flyttalstyper i den arkitektur jag jobbar på. De heter `float` och `double` (i [kursboken](#) kapitel 3.5.1 nämns en tredje sort, `long double`).

Flyttal används för att representera det vi kallar *reella tal* i matematiken. Alltså i praktiken storheter av olika slag, exempelvis temperatur, längd eller tid.

Flyttal representeras i exponentform -- av en mantissa och en exponent. I följande tal är 1,234 mantissa och 3 exponent:

$$1,234 \times 10^3$$

Ju fler siffror i mantissan som flyttalet memorerar, desto större *noggrannhet* eller *precision*. Exponenten bestämmer talets *magnitud*. Ju fler siffror som används till exponenten, desto större till beloppet kan flyttalet bli.

Det heter "flyttal" för att värdet av mantissan är flytande. Värdet beror av storleken på exponenten.

Datorn jobbar binärt, dvs basen för exponenten är inte tio utan två. Och mantissan och exponenten lagras som binära tal.

Följande exempel demonstrerar dels hur inläsning och utmatning ser ut, och dels precision och omfång för de två flyttalstyperna. Jag utnyttjar värden som finns i filen `float.h`, bland annat `FLT_MIN` och `FLT_MAX`. Det framgår i sammanhanget vad värdena betyder.

```
1  #include <stdio.h>
2  #include <float.h>
3
4  main()
5  {
6      float x;
7      double y;
8
9      printf("Två tal: ");
10     scanf("%f %lf", &x, &y);
11
```

```

12     printf("Jag repeterar: %g och %g\n\n", x, y);
13
14     printf("%g <= float <= %g", FLT_MIN, FLT_MAX);
15     printf(" med %d värdesiffror. \n", FLT_DIG);
16     printf("%g <= double <= %g", DBL_MIN, DBL_MAX);
17     printf(" med %d värdesiffror. \n", DBL_DIG);
18 }

```

Så här ser utskriften ut:

```

1  Två tal: 0.00000000000000012345 2.34e234
2  Jag repeterar: 1.2345e-15 och 2.34e+234
3
4  1.17549e-38 <= float <= 3.40282e+38 med 6 värdesiffror.
5  2.22507e-308 <= double <= 1.79769e+308 med 15 värdesiffror.

```

Observera inkonsekvensen: vid inläsning (rad 10) används `%g` och `%lg` för `float` respektive `double`. Men vid utmatning (rad 12) används `%g` för bägge typerna.

Ingen hjälp av datorn att hitta fel

Lägg märke till att tecknet `&` står framför variablerna vid inläsning. Det ska vara så. Glöm dem inte!

Var också noga med att få med alla `%d` både i `scanf` och `printf`. Blanda heller inte ihop specifikationerna med varandra, så att ni till exempel använder `%g` när det ska vara `%d`.

Ett annat vanligt fel är att man bara skriver `d` i stället för `%d`.

Av datorn får ni ofta ingen hjälp att hitta era misstag i `scanf`. Inte ens om felet är uppenbart, till exempel att antalet `%d` inte stämmer.

Programmet körs ändå som om inget har hänt! Det är bara det att resultatet blir obegripligt. Eller plötslig kanske programmet kraschar -- utan att ge ledtrådar om varför.

Kolla därför noga att allt finns med i `scanf`-satsen!

Det här beror på att ni programmerar i C. Det är inte generellt sant för alla programspråk. C är känt för sin ohjälpsamma attityd: *låt programmeraren upptäcka sina egna misstag*.

C är ett svårt språk att lära sig programmera i. Språket är fullt av fällor för nybörjaren att trampa i. Mycket av er tid kommer att gå åt till att leta efter fel som i de flesta andra programspråk inte ens kan uppstå.

I exempelvis Pascal och Ada räknar kompilatorn själv ut vilken typ en variabel har (det står ju i deklarationen!). I dessa språk används varken `%d` eller `&`.

Variabelnamn

Över till något helt annat. Vad får variabler heta? Tydligen får de heta `i`, `j` och `year` i alla fall. Det är ju namn som jag har använt.

Regel: Variabler får innehålla stora och små bokstäver, men bara upp till z. Inga å, ä eller ö. Dessutom får namnet innehålla siffror, och understrykningstecknet `_`. Men namnet får inte börja på en siffra.

Några exempel:

```
■ Alien3 -- Rätt!  
■ Alien 3 -- Fel, inga blanktecken.  
■ Alien_3 -- Rätt!  
■ 20talet -- Fel, ej inleda med siffra  
■ År2001 -- Fel, ej å, ä eller ö.  
■ Year2001 -- Rätt!  
■ _2001 -- Rätt!  
■ Ett_stort_men_fullt_lagligt_variabelnamn --
```

Just det!

Det är skillnad mellan stora och små bokstäver.

Variabelnamnet `year` är inte detsamma som variabelnamnet `Year`.

Det finns i princip ingen gräns för hur långa variabelnamnen får vara. Men ert system kan sätta en gräns.

Nästa lektion

